

ФУНДАМЕНТ IT · ДЕНЬ 18 ИЗ 30

Объектно-ориентированное программирование

Понимаем классы и объекты без сложной теории.

Главная идея

Класс описывает общую форму и поведение объектов. Объект - конкретный экземпляр класса со своими данными. Например, User - класс, а пользователь Анна...

```
class Course:
    def __init__(self, title, days):
        self.title = title
        self.days = days

    def describe(self):
        return f"{self.title}:
{self.days} дней"

course = Course("Фундамент IT", 30)
print(course.describe())
```

План урока

Один блок теории - сразу практика и проверка.

1

Понять смысл

2

Выучить
термины

3

Разобрать код

4

Попробовать
самому

5

Собрать мини-
задачу

Как заниматься

Не читай как книгу. На практических слайдах остановись и попробуй решить сам.

Смысл темы простыми словами

Сначала бытовая идея, потом код.

Идея 1

Класс описывает общую форму и поведение объектов. Объект - конкретный экземпляр класса со своими данными. Например, User - класс, а пользователь Анна - объект.

Идея 2

Атрибут хранит состояние объекта, метод описывает действие. Метод `__init__` инициализирует новый объект и получает его через параметр `self`.

Идея 3

Наследование позволяет расширять класс, но его не следует применять только ради переиспользования пары строк. Часто простые функции и композиция понятнее.

Термины дня

Эти слова нужно узнавать в коде и документации.

class

описание структуры и поведения объектов

object

конкретный экземпляр класса

attribute

данные, принадлежащие объекту

method

функция, определённая внутри класса

constructor

логика начальной настройки нового объекта

inheritance

создание класса на основе поведения другого класса

Код дня

Минимальный пример без лишней сложности.

```
class Course:
    def __init__(self, title, days):
        self.title = title
        self.days = days

    def describe(self):
        return f"{self.title}: {self.days} дней"

course = Course("Фундамент IT", 30)
print(course.describe())
```

Что ищем

Где данные? Где действие? Где результат?

Главный вопрос

Можешь объяснить каждую строку обычным языком?

Разбор кода по шагам

Не переписываем вслепую.

```
steps_day18.py

class Course:
    def __init__(self, title, days):
        self.title = title
        self.days = days

    def describe(self):
        return f"{self.title}: {self.days} дней"

course = Course("Фундамент IT", 30)
print(course.describe())
```

- Прочитай строку сверху вниз.
- Назови данные, с которыми работает код.
- Объясни, какое действие выполняется.
- Проверь, какой результат должен получиться.



Важно

Если строка непонятна - остановись и переведи ее на обычный язык.

Практика: что произойдет?

Ответь без запуска кода.

```
question_day18.py

class Course:
    def __init__(self, title, days):
        self.title = title
        self.days = days

    def describe(self):
        return f"{self.title}: {self.days} дней"

course = Course("Фундамент IT", 30)
print(course.describe())
```

Вопрос

Что программа сделает после запуска?

Подсказка

Ищи последовательность: входные данные -> действие -> результат.

Ответ и объяснение

Сравни со своим рассуждением.

Ожидаемая логика

Этот пример относится к теме: Объектно-ориентированное программирование. Понимаем классы и объекты без сложной теории.

Почему так?

Python выполняет инструкции по порядку. Поэтому важно читать код сверху вниз.

Проверка себя

Если можешь объяснить без подсказки - тема закрепляется.

Практика: сделай руками

Небольшое задание по теме дня.

Задание

Создай класс `Course` с методом `describe()`.

Шаг 1

Создай простой файл или черновик.

Шаг 2

Напиши минимальный рабочий вариант.

Шаг 3

Запусти и проверь результат.

Мини-тест

Выбери ответ и объясни почему.

Вопрос

Что является конкретным экземпляром класса?

A

Модуль

B

Объект

C

Параметр

D

Репозиторий

Ответ мини-теста

Проверяем не память, а понимание.

Правильный ответ

В: Объект

Объяснение

Объект - конкретный экземпляр, созданный по описанию класса.

Частая ошибка новичка

Лучше заранее увидеть, где обычно ломается.

Ошибка

Пытаться запомнить формулировку, но не понимать, где это видно в коде.

Как исправить

После каждого термина придумай свой пример и найди его в коде.

Правило

Термин считается выученным, когда ты можешь объяснить его на бытовом примере.

Самостоятельная мини-задача

Напиши маленький код или схему почти с нуля.

Условие

Создай класс `Course` с методом `describe()`.

- Что программа получает?
- Что она должна сделать?
- Какой результат показать?
- Какие термины дня здесь применяются?

Разбор решения

После попытки сравни с этим подходом.

```
class Course:
    def __init__(self, title, days):
        self.title = title
        self.days = days

    def describe(self):
        return f"{self.title}: {self.days} дней"

course = Course("Фундамент IT", 30)
print(course.describe())
```

Не один вариант

Твой код может отличаться. Главное - он решает задачу и читается.

Проверка

Запусти пример и объясни результат вслух.

Задание со звездочкой

Немного сложнее, но без новой темы.

Условие

Расширь практику дня: добавь проверку ошибки, 2-3 тестовых примера и короткое объяснение для README.

Подсказка

Не усложняй. Сделай маленькое улучшение и обязательно проверь его на примере.

Итог дня

Что должно остаться после урока.

- объяснять тему "Объектно-ориентированное программирование" простыми словами
- узнавать главные термины дня
- читать короткий пример кода
- выполнять практику руками
- находить одну типовую ошибку
- формулировать следующий маленький шаг

Домашнее задание

Создайте класс `Course` с названием и количеством дней. Добавьте метод `describe`, возвращающий понятную строку.